Trust, but Verify: Black-Box Auditing of Opt-Out Compliance in Graph Feature Pipelines

Anonymous Authors

Submitted to the Workshop on Privacy-Preserving Artificial Intelligence (PPAI)

Abstract

Platforms that let users opt out of behavioral data use must ensure that no interaction of an opted-out user influences downstream features. In recommender and marketplace systems, those features are often node embeddings and aggregates derived from a customer–item–merchant interaction graph. We study how an auditor can verify this property when the feature pipeline is a black box, and when the only evidence is a daily sequence of triples $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$: opted-in edges, opted-out edges and produced node features. We cast compliance as the conditional-independence hypothesis $F \perp\!\!\!\perp E_{\text{out}} \mid E_{\text{in}}$, and we note that no black-box procedure can certify zero use: it can only detect leakage above a threshold and bound what remains. We identify the confounders that make naive tests unsound: the correlation between opted-in and opted-out behavior, pipeline stochasticity, rotational non-identifiability of embeddings and receptive-field mismatch. We then propose six tests that exploit the problem's structure: natural canaries, zero-difference tests on invariant neighborhoods, opt-out status-switch difference-in-differences, residual regression with surrogate controls, a membership-inference co-occurrence test with a temporal placebo, and active canaries. Evidence is combined across days with anytime-valid e-values. In a synthetic study, the calibrated tests keep per-day false-positive rates at or below 3% and raise no false alarm in 30 ten-day audits of a clean pipeline. They detect an aggregation bug that leaks 2% of opted-out edges on day one. We also show how two seemingly reasonable test designs produce false-positive rates of 18–44%.

1 Introduction

User-level opt-out is a standard privacy control. Regulations such as the GDPR right to object and the CCPA right to opt out make it a legal obligation as well as a design choice. Large platforms often store behavior as a heterogeneous graph (Zhu et al., 2019; Sarda et al., 2019; Borisyuk et al., 2024). Customers connect to items through views, saves, purchases and reviews, and items connect to the merchants that sell them. Downstream models seldom consume this graph directly. They consume *node-level features*, such as graph embeddings, GNN outputs (Hamilton, Ying, and Leskovec, 2017; Ying et al., 2018; El-Kishky et al., 2022) and hand-crafted aggregates like a restaurant's average preparation time (Hermann and Del Balso, 2017). A feature pipeline produces these features, typically once a day. Section 2 documents this practice.

Honoring an opt-out means that no edge incident to an opted-out customer may influence *any* produced feature. That covers the customer's own feature and the features of every item and merchant reachable through their edges. Filtering bugs break this promise silently: a join that drops the consent predicate, a stale cache, or a training job that reads an unfiltered snapshot.

Why not inspect the pipeline? In principle, a privacy team could review the code or rely on data-lineage metadata. In practice, both fall short. *Velocity.* Gating every pipeline change on manual privacy review slows development. Organizations increasingly let teams ship and verify compliance continuously afterwards. *Least privilege.* Giving one team read access to every pipeline and its data concentrates authority, which creates an insider risk and a security target. An auditor that sees only edge lists, consent labels and outputs needs far less access. *Scale and ownership.* A large platform runs hundreds of feature pipelines in SQL, Spark and notebook code. Different teams own them and change them weekly. Some stages are vendor services or managed feature stores whose source is unavailable. External assessors face the same opacity for intellectual-property reasons. *Code and lineage are not behavior.* Compliance depends on runtime state: which version of the consent table was joined, how fresh a cache was, whether a backfill ran, and which snapshot a training job read. Lineage systems usually track dependencies between tables or columns, not which rows passed a consent predicate. They also miss copies made outside instrumented tools. A code review, finally, judges code at one point in time, and that code keeps changing.

We therefore verify compliance from observable behavior alone, independently of how the pipeline is implemented. This mirrors the separation of duties in financial auditing. The auditor observes only what goes in and what comes out.

We study that setting. Each day the auditor receives $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$ from a fixed but opaque pipeline and must

decide whether F^t was computed from E^t_{in} alone. Our contributions are:

1. A formulation of opt-out compliance as conditional independence, with an explicit limit on what black-box evidence can establish (Section 3).
2. A catalogue of confounders that invalidate naive tests (Section 4).
3. Six complementary audit tests that target aggregation leakage and representation leakage (Section 5).
4. A sequential framework that combines daily evidence with e-values and reports calibrated leakage bounds instead of an unfalsifiable claim of compliance (Section 6).
5. A synthetic study of calibration and power, including two instructive calibration failures (Section 7).

The framing follows empirical auditing of differential privacy (Jagielski, Ullman, and Oprea, 2020; Nasr et al., 2021; Steinke, Nasr, and Jagielski, 2023). In that setting, black-box experiments yield lower bounds on privacy loss, never proofs of privacy. Here they yield detection guarantees above a threshold, never proofs of compliance.

2 The Setting in Practice

Both halves of our setting describe standard industrial practice. The first half is a typed customer–item–merchant interaction graph. The second is a pipeline that turns that graph into node-level features consumed by other models. Table 1 summarizes published systems.

The graph. Public e-commerce logs record exactly our schema. The Tmall repeat-buyer dataset logs each interaction with a user, item, category, merchant and brand, plus one of four action types: click, add-to-cart, purchase or add-to-favorite (Tianchi, 2015). The Taobao UserBehavior dataset uses page views, carts, favorites and purchases (Tianchi, 2018). PyTorch Geometric (Fey and Lenssen, 2019) ships it as a heterogeneous graph with typed user–item edges. Industrial graphs are larger, with the same shape. AliGraph describes a buyer–seller–commodity graph with hundreds of billions of nodes and edges (Zhu et al., 2019). Uber Eats builds user–dish and user–restaurant bipartite graphs whose edges are weighted by order counts and ratings (Sarda et al., 2019). LinkedIn's graph contains tens of node and edge types (Borisyuk et al., 2024).

Node-level features consumed downstream. Two families dominate. The first is *learned embeddings*. PinSage computes embeddings on a graph of 3 billion nodes and 18 billion edges (Ying et al., 2018). Taobao embeds items together with shop and brand side information (Wang et al., 2018). Uber Eats computes a vector for each user, restaurant and dish (Sarda et al., 2019). TwHIN embeddings serve as frozen features in ads ranking, follow recommendation, search ranking and content classification (El-Kishky et al., 2022). AliGraph states that the generated embeddings are fed as features into downstream tasks (Zhu et al., 2019). Airbnb converts listing embeddings into similarity features for its search ranker (Grbovic and Cheng, 2018). The second family is *aggregates*. Uber's Michelangelo computes features such as a restaurant's average meal preparation time over the last seven days. It publishes them to a feature store that is shared across teams and recomputed daily (Hermann and Del Balso, 2017).

Implications for auditing. Three consequences follow. First, one embedding table or aggregate feeds many downstream models, so a single filtering bug spreads widely. The feature boundary is therefore the natural place to audit, not each downstream model. Second, daily batch recomputation produces exactly the daily triples $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$ we assume. Third, real systems deviate from our model in three ways, and none of them breaks it:

- Some systems treat the merchant as side information rather than a node (Wang et al., 2018). Merchant features are then functions of item features, and the audit applies unchanged.
- Some learn from sessions rather than explicit edges (Grbovic and Cheng, 2018; Wang et al., 2018). Sessions are built from the same interactions, so the sessions of opted-out customers belong to E_{out}.
- Near-real-time features (Hermann and Del Balso, 2017) call for triples at a finer time granularity. We leave this extension to future work.

3 Problem Formulation

Setup. On day t, let G^t be a typed bipartite graph between customers $\mathcal{U}$ and items $\mathcal{I}$, plus a static item–merchant map $m : \mathcal{I} \to \mathcal{M}$ and intrinsic item embeddings $x_i \in \mathbb{R}^d$. Each customer u has an opt-out status $o^t_u \in \{0, 1\}$. The edges split into $E^t_{\text{in}} = \{(u, i, \tau) : o^t_u = 0\}$ and $E^t_{\text{out}} = \{(u, i, \tau) : o^t_u = 1\}$, where τ is the interaction type. A fixed pipeline f maps its input to node features $F^t = \{F^t_v\}_{v \in \mathcal{U} \cup \mathcal{I} \cup \mathcal{M}}$. The auditor observes $\mathcal{D}_T = \{(E^t_{\text{in}}, E^t_{\text{out}}, F^t)\}_{t=1}^T$ but neither f nor its internal randomness ξ^t.

Running example. Figure 1 shows a toy marketplace that we reuse throughout the paper. Alice, Carol and Dan are opted in, and Bob is opted out. A compliant pipeline must produce exactly the features it would produce if Bob's three dashed edges did not exist. For example, the Grinder's purchase count must be 1 (Carol), not 2. The Tea scoop must look like an item nobody has bought. Kyoto Crafts, whose only sale came from Bob, must look like a merchant with no sales.

System	Entities in the graph	Interaction edges	Node-level output	Downstream consumer
Uber Eats (Sarda et al., 2019)	users, dishes, restaurants	orders (weighted by count), ratings	GraphSAGE embeddings	candidate generation, personalized ranker
Taobao EGES (Wang et al., 2018)	items (+ shop, brand, category)	co-occurrence in behavior sessions	item embeddings	matching stage of recommendation
AliGraph (Zhu et al., 2019)	buyers, sellers, commodities	user behaviors	GNN embeddings	features for recommendation and search
Pinterest PinSage (Ying et al., 2018)	pins, boards	saves to boards	GCN embeddings	recommendation
Twitter TwHIN (El-Kishky et al., 2022)	users, tweets, advertisers	fave, reply, retweet, follow	HIN embeddings	frozen features in ads, follow, search, safety models
LinkedIn LiGNN (Borisyuk et al., 2024)	members, posts, jobs, companies	engagements, affinities	GNN embeddings	feed, jobs, ads, people recommendation
Airbnb (Grbovic and Cheng, 2018)	users, listings	clicks, bookings (sessions)	listing embeddings	similarity features in search ranking
DoorDash (Koch, 2018)	consumers, stores	orders	store embeddings	store-feed ranking
Uber Michelangelo (Hermann and Del Balso, 2017)	restaurants	orders	aggregates (e.g., 7-day mean prep time)	shared feature store, updated daily
Tmall IJCAI-15 (Tianchi, 2015)	users, items, merchants, brands	click, add-to-cart, purchase, favorite	—	public benchmark
Taobao UserBehavior (Tianchi, 2018)	users, items, categories	pv, cart, fav, buy	—	public benchmark

Table 1: Published systems and datasets that match our setting: typed interaction graphs between customers, items and merchants, turned into node-level embeddings or aggregates that other models consume.

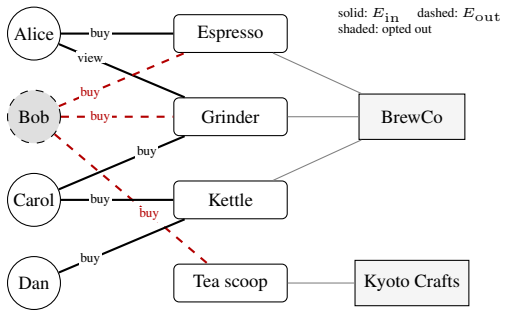


Figure 1: Running example on Monday. Bob has opted out, so his three purchases form E_{out}. The Tea scoop, and with it Kyoto Crafts, was bought only by Bob. Dan will opt out on Tuesday.

Hypothesis. A compliant pipeline satisfies, for some measurable f_0,

$$H_0 : \; F^t = f_0(E^t_{\text{in}}, x, \xi^t) \quad \forall t, \tag{1}$$

with ξ^t independent of E^t_{out} given E^t_{in}. Equivalently, $F^t \perp\!\!\!\perp E^t_{\text{out}} \mid E^t_{\text{in}}, x$. The alternative is that F^t depends on E^t_{out} through some channel. Note that this is a *conditional* statement: F and E_{out} are marginally dependent even under H_0, because opted-in and opted-out customers draw from the same preference distribution.

Leakage modes. We distinguish two channels: *aggregation leakage*, where counts, pooling or message passing include some opted-out edges; and *representation leakage*, where trained parameters (embedding tables, GNN weights, factorization) are fit on data containing opted-out edges, even if inference later uses only E_{in}. The second channel is the harder one to detect. It need not change any feature near an opted-out edge, so tests confined to local neighborhoods can miss it.

What cannot be shown. Could an auditor ever *certify* that a pipeline is compliant? The answer is no, and the reason shapes everything that follows.

Testing H_0 is a conditional-independence (CI) test: we ask whether F still depends on E_{out} once E_{in} is known. Shah and Peters (2020) prove that CI testing is hard in a strong sense. When the conditioning variable is continuous, any test that keeps its false-positive rate at most α under *every* null distribution has power at most α against *every* alternative. Put simply, a test cannot be both always valid and ever useful unless we add assumptions about what the data can look like. Their result concerns general distributions, and our setting has extra structure (a graph and a fixed pipeline), so we do not rely on it directly. A more elementary argument fits our setting and gives a concrete number.

Picture a buggy pipeline f_1 that behaves exactly like a

Challenge	Naive test	Fix
C1 correlation	opted-out pairs close	E_{in} controls, placebo
C2 stochasticity	feature changed	noise-floor control group
C3 rotation	coordinates changed	alignment, invariants
C4 receptive field	1-hop invariance	k-hop invariance
C5 edge identity	untyped invariance	typed edges
C6 opt-out scope	current status on history	fix policy first

Table 2: Six naive tests, the challenge that breaks each one, and the fix used in Section 5.

compliant f_0 except on rare inputs. In Figure 1, f_1 might include Bob's purchases only when a nightly job retries after a timeout and skips the consent join. On every other day the two pipelines produce identical outputs, so no statistic of the outputs can tell them apart on those days.

Proposition 1 (No certificate of zero use)**.** *Let f_0 satisfy H_0, and let f_1 agree with f_0 except on an input event of per-day probability at most δ, on which it uses E_{out} arbitrarily. For any test ϕ of level α against f_0 on T days,* $\Pr_{f_1}[\phi = 1] \leq \alpha + T\delta$.

Proof sketch. Couple the two pipelines on the same inputs and randomness. Their outputs differ only on the union of the per-day events, which has probability at most $T\delta$. Hence the total-variation distance between the laws of $\mathcal{D}_T$ is at most $T\delta$, and the acceptance probabilities differ by at most that much. □

For a concrete sense of scale, suppose the retry bug fires on one day in a thousand ($\delta = 0.001$) and the audit runs for a 30-day quarter. Then *every* level-0.05 test detects it with probability at most $0.05 + 0.03 = 0.08$, however clever the test. More generally, the auditor cannot distinguish "never leaks" from "leaks rarely enough." The same argument covers a deliberately evasive pipeline, which can make its leaking event as rare as it likes. Black-box auditing is therefore a tool against *bugs*, not adversarial pipelines. Bugs tend to be systematic: a missing filter affects every day and many nodes, which is what makes them detectable. The honest output of an audit is a statement of the form "leakage of magnitude at least ε would have been detected with probability at least $1 - \beta$; none was."

4 Why Naive Tests Fail

Each challenge below starts from a natural *naive test* that an auditor might write first. For each one, we give a positive example where the test works, catching a real leak in Figure 1. We then give a counterexample where the same test raises a false alarm on a compliant pipeline, and say what fixes it. Table 2 summarizes the six challenges.

C1: Opted-in and opted-out behavior are correlated.

Naive test. Take every pair of items co-purchased by an opted-out customer. Check whether these pairs are closer in feature space than random item pairs. If so, conclude that opted-out baskets shaped the features.

When it works. Suppose the pipeline trains its embeddings on all edges, including Bob's. Bob is the only link between the Grinder and the Tea scoop, so training pulls them together. The test finds them unusually close and correctly flags the leak.

When it fails. Now take a compliant pipeline. Alice bought the Espresso machine and viewed the Grinder, so the pipeline places these two items close together using her edges alone. Bob also bought both, so the pair counts as an opted-out co-purchase, and the test fires anyway. The underlying problem is that Bob's tastes are not unusual. Popular items are popular in both groups, and similar customers buy similar items whatever their consent status.

Fix. Ask whether F predicts E_{out} *beyond what* E_{in} *already explains*, using controls computed from E_{in} (T4) or a placebo that shares the same confounding (T5).

C2: The pipeline is random.

Naive test. Find items whose opted-out edges changed overnight. If any such item's feature also changed, conclude that the pipeline reads opted-out edges.

When it works. Suppose the pipeline is deterministic and its review count wrongly includes opted-out reviews. Bob reviews the Espresso machine on Tuesday, and its review count goes from 0 to 1. The test sees the change and correctly flags the leak.

When it fails. Now take a compliant pipeline that retrains its embeddings daily from a fresh random initialization. Even run twice on the same input, it gives every item a slightly different embedding, the Espresso machine included. The test fires on noise.

Fix. Compare against a control group of items whose inputs did not change, which measures how much features move for no reason (T2).

C3: Embeddings have no fixed coordinates.

Naive test. For items whose inputs did not change, compare yesterday's and today's embeddings coordinate by coordinate, and flag large differences.

When it works. Suppose retraining is warm-started from yesterday's parameters, so the coordinates are stable from day to day. A leak then shows up as a localized coordinate shift near Bob's items, and the test finds it.

When it fails. Now suppose Tuesday's retraining produces exactly Monday's geometry, but rotated. All distances and angles are unchanged, yet every coordinate of every item differs. The test flags the entire catalog.

Fix. Align the two spaces first, e.g., with orthogonal Procrustes (Schönemann, 1966), or compare only quantities that rotations preserve, such as norms and cosine similarities.

C4: Features depend on more than immediate neighbors.

Naive test. Restrict attention to items whose opted-in *direct* neighbors are unchanged. Their features should then be unchanged, or move only by noise. If they move more when an opted-out edge changes, conclude leakage.
When it works. For a 1-hop aggregate such as a purchase count, the test is exact. The Grinder's opted-in buyers are unchanged, so its count must be unchanged. If Bob buys a second grinder and the count rises, that is a leak.
When it fails. Now suppose the embeddings come from a two-layer GNN. The Grinder's embedding then depends on its buyers *and on everything those buyers bought*. On Tuesday, Carol buys a new teapot. The Grinder's direct neighbors are unchanged, but its embedding legitimately moves, and the test blames Bob.
Fix. Check invariance over the pipeline's true receptive field (k hops), which the auditor may have to estimate (T2). A component fit on the whole graph, such as a matrix factorization, has an unbounded receptive field.

C5: The same pair can change edge type.
Naive test. Represent each item's opted-in neighborhood as the *set of customers* who touched it. If that set is unchanged but a feature changed, conclude leakage.
When it works. If the platform logs a single interaction type, such as purchases only, the customer set fully determines the opted-in input, and the test is sound.
When it fails. With several types, the customer set is not enough. On Monday, Alice viewed the Grinder, and on Tuesday she buys it. The set of customers touching the Grinder is unchanged, but its purchase count correctly rises from 1 to 2. The test sees an unexplained change and blames Bob.
Fix. Track *typed* edges. Section 7 shows that this mistake alone produces a 44% false-positive rate.

C6: The scope of an opt-out must be defined.
Naive test. Build the expected E_{in} by applying each customer's *current* status to all of their history, then run any of the tests above.
When it works. If the pipeline uses only the current day's interactions, history is irrelevant, and current status is the only sensible definition.
When it fails. Suppose the pipeline computes 30-day purchase counts. Dan opts out on Tuesday. His Monday purchase of the Kettle happened while he was opted in. A pipeline that follows a prospective policy keeps that purchase, which is a defensible choice. The naive test, which assumes a retroactive policy, flags it as a leak. The opposite assumption produces the opposite error.
Fix. Agree on the opt-out policy before testing, and derive E_{in} from that definition.

5 Audit Tests

We make one structural assumption, stated for item features; customer and merchant features are analogous.

Test	Targets	Needs	Key control
T1 canary	agg., repr.	long tail	empty $\mathcal{N}^{\text{in}}$
T2 zero-diff	agg., repr.	churn	invariant $\mathcal{N}_k^{\text{in}}$
T3 DiD	agg.	status switches	matched items
T4 residual	agg.	—	surrogate of E_{in}
T5 co-occur.	repr.	—	temporal placebo
T6 active	any	write access	designed canary

Table 3: Audit tests, the leakage modes they target, what they require, and the control that makes them valid under H_0.

Assumption 1 (Locality given global state)**.** *There exist a radius $k \geq 1$ and two fixed functions θ and φ, both unknown to the auditor, such that under H_0, for every node v and day t,*

$$F_v^t = \varphi\big(\mathcal{N}_k^{\text{in}}(v),\, x_v,\, \Theta^t,\, \xi_v^t\big), \qquad \Theta^t = \theta(E_{\text{in}}^t, \xi^t). \quad (2)$$

Here $\mathcal{N}_k^{\text{in}}(v)$ is the typed k-hop neighborhood of v in E_{in}^t, Θ^t is a global state such as trained parameters, and ξ_v^t is the part of the pipeline's randomness ξ^t that acts on node v.

Assumption 1 refines the compliant pipeline f_0 in (1) by splitting it into two stages. The *training* stage θ may look at all of E_{in}^t and produces global state Θ^t. The *readout* stage φ computes each node's feature from that state and the node's own k-hop opted-in neighborhood. For a two-layer GraphSAGE model, for example, θ is the training run, Θ^t are the learned weights, φ is the forward pass, and $k = 2$. For a purchase count, Θ^t is empty, φ counts typed edges, and $k = 1$. Since the pipeline is fixed, θ and φ are the same functions every day and for every node. The tests rely on exactly this: in Figure 1, the Tea scoop's feature is $\varphi(\emptyset, x_{\text{scoop}}, \Theta^t, \xi_{\text{scoop}}^t)$, the same function applied to a never-bought item. The assumption rules out no leakage mode. It only says where, under H_0, information may come from. Table 3 summarizes the tests.

5.1 T1: Natural canaries

Call an item a *natural canary* on day t if every edge incident to it lies in E_{out}^t, i.e., $\mathcal{N}_1^{\text{in}}(i) = \emptyset$ but $\mathcal{N}_1^{\text{out}}(i) \neq \emptyset$. Under Assumption 1, a canary's feature has the same law as that of a *cold* item with no edges at all, conditional on x_i. The test compares canaries with cold items, matched or adjusted on x_i to address C1. It uses a rotation-invariant statistic (C3), here a permutation test on feature norms. Opted-out customers are customer-level canaries. Their features must equal the pipeline's default output, so an exact check applies whenever that default is deterministic. Long-tailed catalogs supply dozens of new canaries every day. Because both groups have the same (empty) E_{in} neighborhood, this test is nearly immune to C1 and C4. In Figure 1, the Tea scoop is a canary. Its purchase count must be 0, and its embedding

must look like that of a never-bought item with similar intrinsic features. Kyoto Crafts is a merchant-level canary, and Bob is a customer-level canary.

5.2 T2: Zero-difference on invariant neighborhoods

T2 compares how much features move overnight for two groups of items whose opted-in inputs did not change. It needs two definitions.

Receptive field. Under Assumption 1, the only opted-in data that can affect F_v^t *locally* is $\mathcal{N}_k^{\text{in}}(v)$: the typed opted-in edges within k hops of v. Everything else reaches v only through the global state Θ^t. We call $\mathcal{N}_k^{\text{in}}(v)$ the *receptive field* of v. In Figure 1, suppose the pipeline is a two-layer GNN ($k = 2$). The Grinder's receptive field then contains its own opted-in edges (Alice's view, Carol's purchase) and the other opted-in edges of those customers (Alice's purchase of the Espresso machine, Carol's purchase of the Kettle). Dan's purchase of the Kettle is three hops away and lies outside it. For a purchase count ($k = 1$), the receptive field is just the item's own opted-in edges.

Opted-out neighborhood. Let $\mathcal{N}^{\text{out}}(v)$ be the set of typed opted-out edges incident to v. The Espresso machine's opted-out neighborhood is $\{(\text{Bob}, \text{buy})\}$. It *changes* between two days when an opted-out customer adds, removes or changes the type of an interaction with the item. If Bob also reviews the Espresso machine on Tuesday, its opted-out neighborhood becomes $\{(\text{Bob}, \text{buy}), (\text{Bob}, \text{review})\}$.

The test. Select items whose receptive field is identical on days t and $t{+}1$. Under H_0, the features of these items can still move, but only because Θ was retrained and because of noise (C2). Those causes affect all selected items alike. Split the selected items into two groups. *Treated* items have a changed opted-out neighborhood, and *control* items do not. A compliant pipeline never reads the opted-out edges, so treated items should move no more than control items. In Figure 1, if Bob reviews the Espresso machine on Tuesday and nothing in its receptive field changes, the Espresso machine is treated. If its embedding moves noticeably more than those of comparable control items, Bob's review leaked.

To implement the test, we align the two days' embeddings (C3), compute each item's relative change $\|\Delta F_i\|/\|F_i\|$, and regress its logarithm on the treatment indicator and log degree. We then test whether the treatment coefficient is positive. For a deterministic 1-hop aggregate, no regression is needed: any change in a treated item's count is a violation, which gives an exact test. Treatment is defined by the item's *own* opted-out edges, because those are the ones a leak affects most. Opted-out changes farther away may also move control items under a leak. That dilutes the contrast and costs power, but it cannot cause false alarms.

5.3 T3: Status-switch difference-in-differences

When customer u switches from opted-in to opted-out, every item u touched should change by exactly u's contribution. For aggregates, that contribution can be computed from the edge list. Comparing these items with matched, unaffected items in a difference-in-differences design (Angrist and Pischke, 2009) detects pipelines that are slow to apply new opt-outs (a stale cache). For example, when Dan opts out on Tuesday, a retroactive policy (C6) requires the Kettle's purchase count to drop from 2 to 1 that same day. If it drops only on Thursday, a cache is stale. Switches back to opt-in are also informative: signal appearing *before* the switch is leakage.

5.4 T4: Residual regression with surrogate controls

Let S_i^{out} be a summary of i's opted-out edges, such as log degree. Under H_0, F_i carries no information about S_i^{out} beyond what E_{in} carries. The auditor can compute its own *surrogate* features $Z_i = g(E_{\text{in}}, x)_i$ (typed counts, a factorization and a propagation of x over E_{in}) and test whether adding F_i to Z_i improves cross-fitted prediction of S_i^{out}, in the spirit of double machine learning (Chernozhukov et al., 2018). The test is valid if Z is rich enough to absorb the dependence between E_{in} and E_{out}, which is precisely C1. In our experiments it is conservative. In Figure 1, S_i^{out} is 1 for the Espresso machine, the Grinder and the Tea scoop, and 0 for the Kettle. The question is whether F predicts these counts better than the auditor's own E_{in}-based features do.

5.5 T5: Co-occurrence membership inference with a temporal placebo

Representation leakage memorizes structure in the parameters (Shokri et al., 2017; Carlini et al., 2022). On graphs, co-interaction is the natural unit (He et al., 2021; Olatunji, Nejdl, and Khosla, 2021). Let $\mathcal{R}^t$ be item pairs that co-occur *only* through opted-out customers today. Their cosine similarity in F^t should not be elevated. In Figure 1, Bob is the only customer who bought both the Grinder and the Tea scoop. If their embeddings are unusually close, Bob's basket has leaked into the parameters. However, such pairs are related by taste (C1): people who buy grinders and tea scoops share interests, whoever they are. A fixed baseline of random non-co-purchased pairs is therefore biased.

We therefore construct a *temporal placebo*. Let $\mathcal{Q}^t$ be pairs that co-occur only through the *fresh* interactions observed on day $t{+}1$, which the day-t pipeline could not have

seen. For example, if Carol buys a teapot and a tea tin together on Tuesday, that pair enters Monday's placebo set. Both sets are draws from the same behavior model under identical exclusions: neither pair set co-occurs in E_{in}^t, and each excludes the other source. They therefore share the confounding of C1. We regress similarity on group membership, adjusting for degree, and test the group effect. Pairs are subsampled to be item-disjoint, because overlapping pairs share items and otherwise make the standard errors anti-conservative. Section 7 shows that both the symmetric exclusion and the disjointness are necessary.

5.6 T6: Active canaries

If the auditor can write to the system, it can enroll synthetic opted-out accounts whose behavior is drawn from a secret random design. For example, an opted-out test account "Eve" buys a secretly chosen random set of long-tail items, and so creates canaries like the Tea scoop on purpose. This mirrors canary insertion in memorization and DP auditing (Carlini et al., 2019; Jagielski, Ullman, and Oprea, 2020; Steinke, Nasr, and Jagielski, 2023). Because the design is randomized, T1 and T5 become randomization tests whose validity rests on the auditor's own coin flips, not on behavioral exchangeability. The number of canaries directly controls power.

6 Sequential Evidence and Reporting

E-values. Audits run continuously, so stopping at the first small p-value inflates the type-I error. We convert each daily p-value p_t into an e-value with the calibrator $e_t = \kappa p_t^{\kappa-1}$ ($\kappa = 1/2$) (Vovk and Wang, 2021), and we track the e-process $M_T = \prod_{t \le T} e_t$. If each e_t is valid conditional on the past, M_T is a test supermartingale. Ville's inequality then gives $\Pr_{H_0}[\exists T : M_T \ge 1/\alpha] \le \alpha$, so the auditor may monitor indefinitely and stop at any time (Shafer et al., 2011; Ramdas et al., 2023). Daily graphs share customers, so conditional validity is an assumption. Sample splitting by customer across days, or using only fresh canaries each day, makes it more plausible. Several tests are combined by averaging their e-processes, which needs no multiplicity correction.

Reporting bounds. The auditor should report a detectable effect size, not "no leakage." For a leakage family indexed by ρ (for example, the fraction of opted-out edges used), one injects ρ-leakage into a *surrogate* pipeline built on the observed data. One then calibrates each test's power curve and reports the smallest ρ that would have been detected within T days with probability $1-\beta$. With active canaries (T6), this calibration runs on the real pipeline.

7 Experiments

Simulator. We simulate 3,000 customers, 2,000 items with intrinsic embeddings $x_i \in \mathbb{R}^{16}$ and Zipf popularity (exponent 1.1), and 100 merchants. Each customer draws $1 + \text{Poisson}(7)$ items with probability proportional to popularity times $\exp(2\, p_u^\top x_i/4)$. Edge types are view, save, purchase and review, with probabilities $0.60/0.15/0.17/0.08$. Each day, 20% of customers redraw their interactions. Opt-out status follows a two-state Markov chain with stationary rate 10% (in→out 0.005, out→in 0.045). Each audit spans 10 days.

Pipelines. Item features concatenate (i) typed counts and the number of unique customers (aggregation component) with (ii) a rank-16 randomized SVD of the type-weighted interaction matrix (Halko, Martinsson, and Tropp, 2011) and (iii) a propagation of x over the item–item co-interaction graph, in the style of SIGN (Frasca et al., 2020) (representation component). Customer features are typed counts and the mean embedding of their items. The *clean* pipeline uses E_{in} everywhere. The *aggregation-leak* pipeline adds a fraction ρ of opted-out edges, selected by a fixed hash, to (i) and customer pooling only. The *representation-leak* pipeline adds them to (ii) and (iii) only. The SVD is reseeded daily, so all pipelines are stochastic.

Tests. We run T1, T2 (k=1 for aggregates and k=2 for embeddings), T4 (ridge, 5-fold cross-fitting, with surrogate Z = typed log-counts, x, a rank-32 SVD of E_{in} and a propagation over E_{in}) and T5. T3 overlaps with T2 in this simulator, and T6 needs write access, so we omit both. A typical day yields 34 canaries against 210 cold items, 40 treated and 406 control items for T2, and 298 real against 412 placebo item-disjoint pairs for T5.

Calibration. Over 30 clean audits (300 test-days), the per-day rejection rates at $\alpha = 0.05$ are 1% (T1), 3% (T2), 0% (T4) and 2% (T5). No e-process, individual or averaged, crosses $1/\alpha = 20$ in any audit.

Two failures along the way are worth reporting, because both look reasonable. First, defining T2's invariance on untyped (customer, item) pairs yields a **44%** false-positive rate. Resampled customers often re-pick the same item with a different type, which legitimately changes typed counts (C5). Second, building T5's placebo from only the *new* edges of day t+1, with pair-level OLS errors, yields **18–20%**. New edges are biased away from items a customer already had, and overlapping pairs violate independence. Both failures are confounding, not lack of power, and neither is visible without a null study.

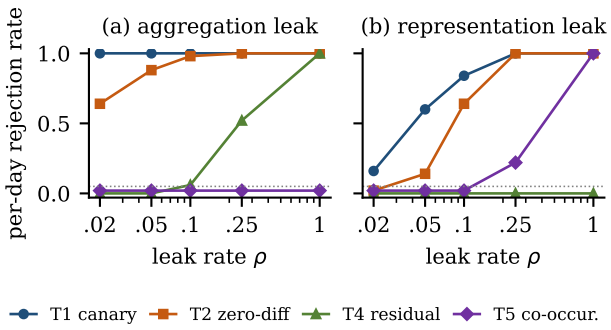


Figure 2: Per-day rejection rate ($\alpha = 0.05$) against the fraction ρ of opted-out edges leaked, averaged over 5 seeds $\times$ 10 days. Dotted line: nominal level. T4 targets aggregates only, and T5 targets representation leakage only.

	leak rate ρ				
Pipeline	.02	.05	.10	.25	1.0
Aggregation leak	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)
Representation leak	0/5	3/5 (3)	5/5 (2)	5/5 (1)	5/5 (1)
Clean (ρ=0)	0/30 false alarms				

Table 4: Audits in which the averaged e-process over T1, T2, T4 and T5 reached 20 within 10 days (median detection day in parentheses).

Power. Figure 2 and Table 4 summarize the results. For aggregation leakage, T1 alone rejects on every day even at $\rho = 0.02$, which leaks roughly 0.2% of all edges. An opted-out customer with any leaked edge has a non-default feature, and the exact check catches it. T2's exact aggregate check follows closely. T4 needs $\rho \geq 0.25$: summarizing E_{out} by degree throws away the identity of the leaked edges. Representation leakage is harder. The combined e-process detects it on day 2 at $\rho = 0.1$, and in 3 of 5 audits at $\rho = 0.05$. T1 is again the most powerful single test, because the factorization assigns a nonzero embedding to canary items whose training column is no longer empty. T5 detects only $\rho \geq 0.25$. It remains valuable, though, because it does not depend on long-tail canaries or invariant neighborhoods, which dense graphs and merchant-level features may lack.

8 Discussion and Related Work

Relation to DP auditing. DP auditing inserts canaries and measures distinguishability to lower-bound ε (Jagielski, Ullman, and Oprea, 2020; Nasr et al., 2021; Steinke, Nasr, and Jagielski, 2023). Our tests can be read as auditing a degenerate mechanism that should have $\varepsilon = 0$ with respect to E_{out}. Two differences matter. The auditor usually cannot choose the inputs, hence the emphasis on *natural* experiments (canaries, invariant neighborhoods, status switches, temporal placebos). And the reference distribution must be built from correlated behavioral data, not from a known mechanism.

Relation to unlearning. Verifying that data was removed from a model is studied under machine unlearning (Ginart et al., 2019; Bourtoule et al., 2021), typically with membership inference. Opt-out compliance is a continuous, graph-structured version of that problem. Removal applies every day, and the influence of removed data spreads through neighbors.

Graph privacy attacks. Link-stealing and GNN membership inference (He et al., 2021; Olatunji, Nejdl, and Khosla, 2021) show that node outputs reveal edges. T5 turns this attack into an audit statistic and adds a placebo to separate memorization from homophily.

Limitations. Simulations cannot establish calibration on real behavioral data. Before deployment, every test should first be run on days or slices where compliance is known, such as a pipeline replica fed only E_{in}. Assumption 1 and the exchangeability behind T5's placebo are modeling choices. Seasonality or product launches can break them, and T6 is the remedy when write access exists. Daily e-values are not strictly independent. Finally, Proposition 1 bounds what any of this can achieve against a pipeline that leaks deliberately and sparsely. Code audits, data lineage and attested execution remain necessary complements.

9 Conclusion

Opt-out compliance in black-box graph feature pipelines cannot be proven from outputs, but it can be falsified efficiently. The most effective evidence comes from nodes whose correct output is known in advance: natural canaries and invariant neighborhoods. Validity, however, depends on details: typed edges, the true receptive field, symmetric placebos and dependence between test units. Every audit should begin with a null study, and its result should be reported as a calibrated detection bound.

References

Angrist, J. D., and Pischke, J.-S. 2009. *Mostly Harmless Econometrics*. Princeton University Press.

Borisyuk, F.; He, S.; Ouyang, Y.; Ramezani, M.; Du, P.; et al. 2024. LiGNN: Graph neural networks at LinkedIn. In *KDD*.

Bourtoule, L.; Chandrasekaran, V.; Choquette-Choo, C. A.; et al. 2021. Machine unlearning. In *IEEE Symposium on Security and Privacy*.

Carlini, N.; Liu, C.; Erlingsson, Ú.; Kos, J.; and Song, D. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *USENIX Security Symposium*.

Carlini, N.; Chien, S.; Nasr, M.; Song, S.; Terzis, A.; and Tramèr, F. 2022. Membership inference attacks from first principles. In *IEEE Symposium on Security and Privacy*.

Chernozhukov, V.; Chetverikov, D.; Demirer, M.; Duflo, E.; Hansen, C.; Newey, W.; and Robins, J. 2018. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal* 21(1).

El-Kishky, A.; Markovich, T.; Park, S.; Verma, C.; Kim, B.; Eskander, R.; Malkov, Y.; Portman, F.; Samaniego, S.; Xiao, Y.; and Haghighi, A. 2022. TwHIN: Embedding the Twitter heterogeneous information network for personalized recommendation. In *KDD*.

Fey, M., and Lenssen, J. E. 2019. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

Frasca, F.; Rossi, E.; Eynard, D.; Chamberlain, B.; Bronstein, M.; and Monti, F. 2020. SIGN: Scalable inception graph neural networks. *arXiv:2004.11198*.

Grbovic, M., and Cheng, H. 2018. Real-time personalization using embeddings for search ranking at Airbnb. In *KDD*.

Ginart, A.; Guan, M.; Valiant, G.; and Zou, J. 2019. Making AI forget you: Data deletion in machine learning. In *NeurIPS*.

Halko, N.; Martinsson, P.-G.; and Tropp, J. A. 2011. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53(2).

Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. In *NeurIPS*.

Hermann, J., and Del Balso, M. 2017. Meet Michelangelo: Uber's machine learning platform. Uber Engineering Blog. https://www.uber.com/blog/michelangelo-machine-learning-platform/.

He, X.; Jia, J.; Backes, M.; Gong, N. Z.; and Zhang, Y. 2021. Stealing links from graph neural networks. In *USENIX Security Symposium*.

He, X.; Deng, K.; Wang, X.; Li, Y.; Zhang, Y.; and Wang, M. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *SIGIR*.

Jagielski, M.; Ullman, J.; and Oprea, A. 2020. Auditing differentially private machine learning: How private is private SGD? In *NeurIPS*.

Koch, M. 2018. Personalized store feed with vector embeddings. DoorDash Engineering Blog.

Nasr, M.; Song, S.; Thakurta, A.; Papernot, N.; and Carlini, N. 2021. Adversary instantiation: Lower bounds for differentially private machine learning. In *IEEE Symposium on Security and Privacy*.

Olatunji, I. E.; Nejdl, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *IEEE TPS-ISA*.

Ramdas, A.; Grünwald, P.; Vovk, V.; and Shafer, G. 2023. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* 38(4).

Sarda, A.; Jain, A.; Liu, I.; et al. 2019. Food discovery with Uber Eats: Using graph learning to power recommendations. Uber Engineering Blog. https://www.uber.com/blog/uber-eats-graph-learning/.

Schönemann, P. H. 1966. A generalized solution of the orthogonal Procrustes problem. *Psychometrika* 31(1).

Shafer, G.; Shen, A.; Vereshchagin, N.; and Vovk, V. 2011. Test martingales, Bayes factors and p-values. *Statistical Science* 26(1).

Shah, R. D., and Peters, J. 2020. The hardness of conditional independence testing and the generalised covariance measure. *The Annals of Statistics* 48(3).

Shokri, R.; Stronati, M.; Song, C.; and Shmatikov, V. 2017. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy*.

Steinke, T.; Nasr, M.; and Jagielski, M. 2023. Privacy auditing with one (1) training run. In *NeurIPS*.

Alibaba Tianchi. 2015. Repeat buyers prediction after sales promotion (IJCAI-15 competition dataset). https://tianchi.aliyun.com/.

Alibaba Tianchi. 2018. User behavior data from Taobao for recommendation. https://tianchi.aliyun.com/dataset/649.

Vovk, V., and Wang, R. 2021. E-values: Calibration, combination and applications. *The Annals of Statistics* 49(3).

Wang, J.; Huang, P.; Zhao, H.; Zhang, Z.; Zhao, B.; and Lee, D. L. 2018. Billion-scale commodity embedding for e-commerce recommendation in Alibaba. In *KDD*.

Ying, R.; He, R.; Chen, K.; Eksombatchai, P.; Hamilton, W. L.; and Leskovec, J. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*.

Zhu, R.; Zhao, K.; Yang, H.; Lin, W.; Zhou, C.; Ai, B.; Li, Y.; and Zhou, J. 2019. AliGraph: A comprehensive graph neural network platform. *PVLDB* 12(12).